

AWS Services Exhaustive Guide for Senior Backend Interviews

This guide is focused on practical architecture, trade-offs, and incident-driven discussion.

1) Core AWS Building Blocks for Backend Platforms

1.1 Compute Choices

EC2:

- Full control, custom runtimes, long-lived workloads.
- Requires patching, scaling orchestration, and AMI lifecycle.

ECS (Fargate/EC2):

- Container orchestration with lower operational overhead than raw EC2.
- Fargate avoids node management.

EKS:

- Managed Kubernetes control plane with cloud-native integrations.
- Best when you need k8s portability and advanced orchestration features.

Lambda:

- Event-driven serverless compute.
- Excellent for bursty and short tasks, but watch cold starts and execution limits.

Interview decision framing:

- Stable high-throughput API -> ECS/EKS/EC2.
- Event processing with burst traffic -> Lambda.
- Mixed workloads -> combine services by workload profile.

1.2 Storage Choices

S3:

- Durable object storage.
- Use lifecycle transitions and versioning.

EBS:

- Block storage for EC2.
- Understand gp3/io2, IOPS, throughput limits.

EFS:

- Shared POSIX filesystem across instances.
- Good for shared artifacts and stateful workloads needing file semantics.

2) Database Services Deep Dive

2.1 RDS and Aurora

RDS engines:

- Managed Postgres, MySQL, MariaDB, SQL Server, Oracle.

Aurora:

- Cloud-optimized relational engine.
- Storage decoupled from compute, fast failover, reader endpoints.

Must-know capabilities:

- Multi-AZ for HA.
- Read replicas for read scaling.
- Automated backups and PITR.
- Parameter groups and option groups.

Interview trade-offs:

- RDS Postgres: compatibility and control.
- Aurora Postgres: better managed failover/scaling in many scenarios, higher cost.

2.2 DynamoDB

Concepts:

- Partition key and sort key design.
- Access-pattern-first table design.
- GSI/LSI for secondary access.

Operational topics:

- Provisioned vs on-demand capacity.
- Hot partition avoidance.
- Conditional writes for idempotency and optimistic locking.
- TTL and streams.

When to pick:

- Massive scale key-value/document patterns with predictable access paths.

2.3 ElastiCache

Redis/Memcached managed clusters:

- Redis for rich data structures and persistence options.
- Memcached for simple in-memory caching.

Key concerns:

- eviction policy tuning.
- cache stampede handling.
- key schema and memory fragmentation.

3) Messaging and Eventing on AWS

3.1 SQS

Features:

- Standard queue: high throughput, at-least-once delivery.
- FIFO queue: ordering + dedup windows, lower throughput.

Operational tuning:

- Visibility timeout should exceed processing duration.
- DLQ for poison messages.
- Long polling to reduce empty receives and cost.

3.2 SNS

- Pub-sub fanout.
- Common pattern: SNS topic -> multiple SQS queues.
- Filter policies for selective subscription.

3.3 EventBridge

- Event bus for service integration.
- Supports routing by event pattern.
- Useful for SaaS integrations and event-driven architectures.

3.4 Amazon MSK (Kafka)

- Managed Kafka clusters.
- Integrates with IAM auth in some setups, monitoring, and broker ops.
- Still requires topic/partition/capacity design ownership.

4) Networking and Traffic Management

4.1 VPC Fundamentals

- Public/private subnets.
- Route tables and NAT gateways.
- Security groups (stateful) vs NACLs (stateless).

Senior interview must-cover:

- Why backend services usually sit in private subnets.
- How egress is controlled.

4.2 Load Balancers

ALB:

- HTTP/HTTPS routing, host/path-based rules.

NLB:

- Layer 4 high-performance TCP/UDP.

GWLB:

- Appliance insertion pattern.

4.3 API Gateway

- Managed API front door.
- Auth, throttling, usage plans, transformations.
- Strong for serverless and external APIs.

5) Deployment and Release Strategies on AWS

5.1 CI/CD

Common stack:

- CodePipeline + CodeBuild + CodeDeploy.
- Or GitHub Actions/GitLab + Terraform/CloudFormation.

5.2 Deployment Patterns

- Rolling deployment.
- Blue/green deployment.
- Canary with weighted target groups.

For interview impact:

- Mention rollback trigger signals: error rate, p95, saturation metrics.

5.3 Immutable Infrastructure

- Bake AMIs/images.
- Avoid in-place drift.
- Traceable rollbacks by artifact version.

6) Scaling Strategies

6.1 Horizontal and Vertical Scaling

- ASG target tracking on CPU, request count, custom metrics.
- ECS service auto scaling.
- EKS HPA/VPA and Cluster Autoscaler.

6.2 Database Scaling

- Read replicas.
- Partitioning/sharding where needed.
- Connection pooling with RDS Proxy.

6.3 Queue-Based Smoothing

- Use SQS/Kafka to decouple producers/consumers.
- absorb burst traffic and process at steady rate.

7) Observability and Operations on AWS

7.1 CloudWatch

- Metrics, logs, alarms, dashboards.
- Custom metrics for business and reliability KPIs.

Important metrics examples:

- API: p50/p95/p99 latency, 4xx/5xx rates.
- Queue: age of oldest message, DLQ depth.
- DB: CPU, storage, connections, replication lag.

7.2 Tracing

- AWS X-Ray or OpenTelemetry + collector pipeline.
- Use trace IDs across services and logs.

7.3 Incident Workflow

- Alert -> triage -> mitigation -> root cause -> prevention.
- Use runbooks and postmortems with action owners.

8) Security and Governance

8.1 IAM Design

- Least privilege policies.
- Role-based access for workloads.
- Avoid static credentials.

8.2 Secrets and Encryption

- AWS Secrets Manager/SSM Parameter Store.
- KMS for envelope encryption.
- TLS in transit and encryption at rest.

8.3 Network Security

- Private endpoints/VPC endpoints where possible.
- WAF and Shield for internet-facing apps.

9) Disaster Recovery and Business Continuity

DR patterns:

- Backup and restore.
- Pilot light.
- Warm standby.
- Multi-site active-active.

Interview detail:

- define RTO and RPO explicitly.
- prove recovery drills, not just configuration existence.

10) Cost Optimization Talking Points

- Right-size compute and storage.
- Use Savings Plans/Reserved Instances for steady workloads.
- S3 lifecycle and compression strategies.
- Efficient log retention policies.
- Spot for fault-tolerant batch jobs.

11) Example Architecture 1: High-Scale API Platform

Flow:

- CloudFront -> ALB -> ECS/EKS services.
- RDS Postgres primary + read replicas.
- Redis cache via ElastiCache.
- Kafka/MSK or SQS for async workloads.
- CloudWatch + tracing + centralized logs.

Key resilience controls:

- timeout budgets + retries with jitter.
- rate limiting.
- circuit breakers.
- graceful degradation.

12) Example Architecture 2: Event-Driven Processing

Flow:

- API writes command to DB + outbox.
- Outbox publisher emits event to SNS/EventBridge/Kafka.
- Consumers process and write status updates.
- DLQ and replay tooling for failed events.

Interview win:

- demonstrate idempotent consumers and schema versioning strategy.

13) Rapid-Fire AWS Interview Questions

1. Why choose SQS over Kafka?
 - SQS for simple managed queue semantics and low ops.
 - Kafka for high-throughput event streaming, replay, and partitioned ordering.
2. When would you choose Aurora over RDS Postgres?
 - Need faster failover/managed scaling characteristics with PostgreSQL compatibility requirements.
3. How do you secure service-to-service communication?
 - IAM roles, mTLS/TLS, private networking, and scoped security groups.
4. How do you avoid Lambda cold-start impact?
 - Provisioned concurrency, smaller packages, optimized init path, workload splitting.
5. How do you design multi-region failover?
 - Data replication strategy, DNS failover, stateless app tier, validated runbooks and drills.

14) Senior Interview Story Prompts (AWS)

Prepare 5 stories:

- Major cost optimization initiative.
- Large-scale production migration.
- DR test and failover validation.
- Incident involving queue backlog and recovery.
- Security hardening and least-privilege rollout.

Include metrics and business impact in every story.

15) Hands-On Snippets and Interview Explanations

15.1 SQS Queue with DLQ (CloudFormation)

Resources:

[OrdersDLQ:](#)

```

    Type: AWS::SQS::Queue
    Properties:
      QueueName: orders-dlq

OrdersQueue:
  Type: AWS::SQS::Queue
  Properties:
    QueueName: orders-main
    VisibilityTimeout: 60
    RedrivePolicy:
      deadLetterTargetArn: !GetAtt OrdersDLQ.Arn
      maxReceiveCount: 5

```

Interview explanation:

- Visibility timeout must exceed processing time.
- DLQ prevents poison messages from blocking healthy traffic.

15.2 ECS Service Auto Scaling (CLI)

```

aws application-autoscaling register-scalable-target \
  --service-namespace ecs \
  --resource-id service/prod-cluster/orders-api \
  --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 --max-capacity 20

aws application-autoscaling put-scaling-policy \
  --service-namespace ecs \
  --resource-id service/prod-cluster/orders-api \
  --scalable-dimension ecs:service:DesiredCount \
  --policy-name orders-cpu-target \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration
'{"TargetValue":60.0,"PredefinedMetricSpecification":
{"PredefinedMetricType":"ECSServiceAverageCPUUtilization"}}'

```

Interview explanation:

- Target tracking keeps service near utilization target.
- Min/max guardrails prevent cost blowouts and under-provisioning.

15.3 CloudWatch Alarm for p95 Latency

```

aws cloudwatch put-metric-alarm \
  --alarm-name orders-api-p95-latency \
  --namespace "Custom/API" \
  --metric-name p95LatencyMs \
  --statistic Average \
  --period 60 \
  --evaluation-periods 5 \

```

```
--threshold 350 \  
--comparison-operator GreaterThanThreshold
```

Interview explanation:

- Multi-window evaluation avoids noisy alerts from single spikes.

15.4 IAM Policy Snippet (Least Privilege)

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": ["s3:GetObject"],  
      "Resource": ["arn:aws:s3:::orders-bucket-prod/*"]  
    }  
  ]  
}
```

Interview explanation:

- Scope action and resource narrowly; avoid wildcard grants.

15.5 RDS Proxy Use Case

Example pattern:

- Application pods use RDS Proxy endpoint.
- Proxy pools and reuses DB connections.
- Burst traffic does not translate to equal DB session spike.

Interview explanation:

- This is a common fix for connection exhaustion incidents.

16) L4 Architecture Drill (AWS Official + Production Patterns)

16.1 Well-Architected Decision Framework

For every design answer, score choices against:

- Operational Excellence
- Security
- Reliability
- Performance Efficiency
- Cost Optimization
- Sustainability

Interview expectation:

- State which pillar you are intentionally trading off and why.

16.2 Multi-Region Strategy Matrix

- Active-passive: simpler ops, slower failover, lower cost.

- Active-active: better availability and latency, hard consistency and conflict handling.
- Pilot light/warm standby: disaster recovery middle ground.

RTO/RPO framing:

- RTO: how fast service restores.
- RPO: how much data loss window is tolerated.

16.3 Secure-by-Default Baseline

- IAM least privilege with scoped resources and conditions.
- KMS encryption for data at rest, TLS for data in transit.
- Secrets Manager/SSM Parameter Store for credential lifecycle.
- CloudTrail + Config + GuardDuty for audit and detection.

16.4 Event-Driven Reference for Scale

```
API Gateway -> Lambda/ECS -> SQS (buffer) -> workers -> DynamoDB/RDS
                                     -> DLQ for poison events
```

Interview explanation:

- Queue buffering protects downstream during bursts and incidents.

16.5 Cost and Performance Checklist

- Right-size compute with autoscaling boundaries.
- Push static content to CloudFront.
- Use S3 lifecycle for log/archive buckets.
- Prefer Graviton where workload-compatible.
- Track unit economics (cost/request, cost/tenant, cost/GB streamed).

16.6 Senior Grilling Questions

1. When do you pick Aurora vs DynamoDB for an orders domain?
2. How do you design zero-downtime secret rotation?
3. How would you detect and stop retry storms across SQS consumers?
4. How do you prevent a cross-AZ dependency from becoming hidden SPOF?
5. What goes in your game-day for regional failover?

17) L5 AWS Decision Matrix (Service Selection Under Constraints)

17.1 Compute Selection Under Real Constraints

Use this interview matrix:

- Lambda: bursty/event-driven, short-lived compute, minimal ops overhead.
- ECS Fargate: container portability with less infra ownership.
- EKS: platform-level standardization, multi-team Kubernetes contracts.
- EC2/ASG: custom runtime/networking and tight cost/perf tuning.

Discuss trade-offs explicitly:

- startup latency vs steady-state efficiency,
- platform complexity vs team maturity,

- operational burden vs feature velocity.

17.2 Data Plane Selection

- DynamoDB: key-value access at extreme scale, strict access-pattern-first modeling.
- Aurora/Postgres: relational constraints, complex joins/transactions.
- S3 + Athena/Glue: analytical and archival data paths.
- ElastiCache: latency shield and hot-path offload.

17.3 Messaging Selection

- SQS for queueing and buffering.
- SNS for fanout notifications.
- EventBridge for event routing across bounded contexts.
- Kinesis for ordered stream ingest and replay windows.

Interview signal:

- "I choose by failure mode and operating model, not by feature checklist only."

18) AWS Architect Incident Playbooks

18.1 API Latency Spike After Deploy

Triage flow:

1. ALB target response time and 5xx split.
2. App-level p95/p99 and dependency spans.
3. DB connection saturation and lock waits.
4. Rollback/canary stop if burn rate exceeds SLO budget.

18.2 Regional Degradation

- Verify blast radius: single AZ, regional, or control-plane dependency.
- Shift read traffic to secondary region if strategy supports it.
- Protect write path with queue buffering and explicit degradation mode.

18.3 Cost Explosion During Traffic Event

- Detect top cost drivers: NAT, data transfer, over-provisioned compute, logs.
- Apply short-term controls: autoscaling guardrails, TTL, sampling.
- Apply structural fixes: rightsizing, reserved plans, architecture change.

19) L5 Grilling Questions (AWS)

1. How do you prove your multi-region strategy meets RTO/RPO targets?
2. When does active-active increase risk rather than reduce it?
3. How do you stop retries at three layers from causing retry storms?
4. Why can IAM least privilege still fail in cross-account eventing?
5. How do you audit and enforce architecture standards at org scale?

20) 60-Minute AWS Architect Revision Sprint

0-20 min:

- Well-Architected pillars and trade-off language.

20-40 min:

- Compute/data/messaging decision matrices with failure modes.

40-60 min:

- One migration story + one incident story with metrics.
-